



Faculty of Computers and artificial intelligence

Cairo University

CS352 - Advanced Software Engineering

Restaurant Ordering & Billing System

Submit to	Dr. Manar Elkady
Name	Roa Mohamed & Mennat-Allah Abdallah
ID	20230142 & 20231178

November of 2025

1. Overview of the system

This system is designed to be **modular, extensible, and maintainable**, adhering to SOLID principles and employing several design patterns. The goal is to support multiple menu families, customizable food items, flexible order workflows (dine-in, takeaway, delivery), various payment methods, discount policies, billing, and internal notifications.

2. Design Patterns Used

2.1 Composite Pattern – Menu + Menu Items

Why used:

The restaurant has hierarchical menus:

- Menu Family → Category (Pizza/Burger/Drinks) → Items
The Composite Pattern enables treating menus and individual menu items the same way.

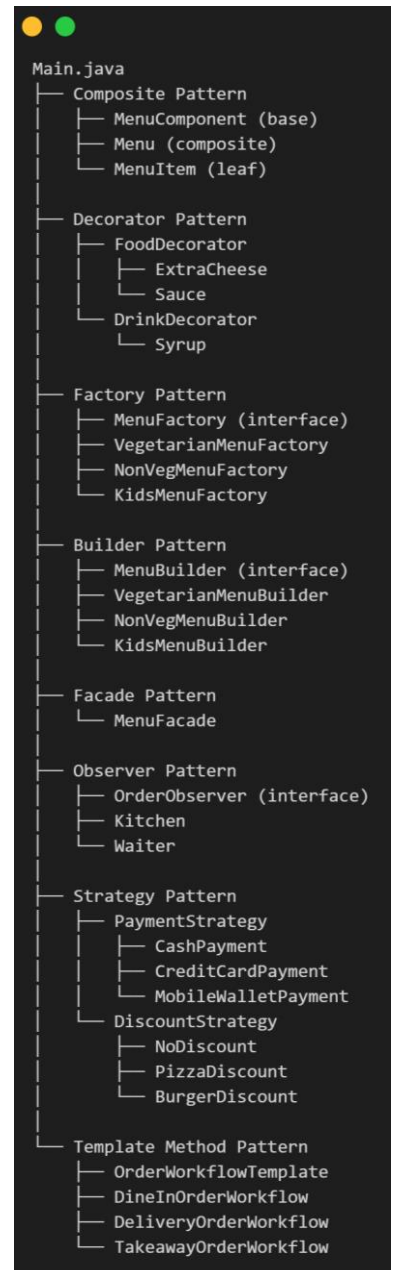
How applied:

- MenuComponent = abstract base component
- Menu = composite (can contain children)
- MenuItem = leaf node
This enables printing the full menu recursively and grouping categories flexibly.

2.2 Decorator Pattern – Add-ons (Extra Cheese, Sauce, Syrup)

Why used:

Customers can customize items with extra toppings without modifying existing classes.



How applied:

- FoodDecorator and DrinkDecorator wrap a base item.
- Add-ons dynamically increase price and description.
- Keeps open for extension but closed for modification (OCP).

2.3 Abstract Factory Pattern – Menu Families (Veg / Non-Veg / Kids)

Why used:

Each menu family provides its own pizzas, burgers, drinks.

We must ensure **consistent product creation** (e.g., vegetarian family only creates vegetarian items).

How applied:

- MenuFactory interface defines creation methods.
- Concrete factories: VegetarianMenuFactory, NonVegMenuFactory, KidsMenuFactory.

2.4 Builder Pattern – Assembling a Complete Menu

Menus need to be built category by category.

Why used:

To separate **product creation (factory)** from **menu assembly (builder)**.

How applied:

- MenuBuilder interface and concrete builders create full menus:
 - Pizza section
 - Burger section
 - Drinks section
- Each builder uses its factory internally.

2.5 Facade Pattern – MenuFacade

Why used:

The main program should not know:

- Which factory to use
- How the builder works
- How the menu structure is assembled

MenuFacade.createFamilyMenu() hides all the complexity.

2.6 Strategy Pattern – Payment Methods

Why used:

Different payment methods follow the same interface but different processing.

How applied:

- PaymentStrategy implemented by:
 - CashPayment
 - CreditCardPayment
 - MobileWalletPayment

2.7 Strategy Pattern – Discounts

Why used:

Different promotions (pizza discount, burger discount, no discount) require flexible rules.

How applied:

- DiscountStrategy with implementations:
 - NoDiscount
 - PizzaDiscount
 - BurgerDiscount

2.8 Observer Pattern – Kitchen & Waiter Notifications

Why used:

When an order is placed, different departments need to be notified.

How applied:

- Order = Subject
- Observers = Kitchen, Waiter
- When order is placed → observers receive updates.

2.9 Template Method Pattern – Order Workflow

Why used:

All order types follow the same high-level steps:

1. Display menu
2. Select items
3. Pre-payment tasks
4. Process payment
5. Notify observers

How applied:

- OrderWorkflowTemplate defines the skeleton.
- Concrete workflows override different steps:
 - DineInOrderWorkflow
 - TakeawayOrderWorkflow
 - DeliveryOrderWorkflow

3. SOLID Principles

S – Single Responsibility

- Each class handles only one job (e.g., PizzaDiscount only calculates pizza discounts).

O – Open/Closed

- New menu families, discounts, or add-ons can be added without modifying existing code (factories, decorators, strategies).

L – Liskov Substitution

- All menu items, pizzas, burgers, drinks behave like MenuComponent or MenuItem.

I – Interface Segregation

- Small focused interfaces: PaymentStrategy, DiscountStrategy, OrderSubject.

D – Dependency Inversion

- Main program depends on abstractions (factories, strategies, workflows), not concrete classes.